

On-Demand Portfolio

AWS Serverless Architecture Documentation

Sayyad Sameer

sayyadsameerm3@gmail.com | github.com/Sayyadsameer

Last Updated: July 2026 | Version 2.0

Table of Contents

- 1. Project Overview
- 2. Objectives
- 3. Project Structure
- 4. Architecture & Workflow
- 5. AWS Services Used
- 6. Security & Architectural Rationale
- 7. Features
- 8. Implementation Steps
 - 8.1 S3 Setup
 - 8.2 IAM Roles & Policies
 - 8.3 Lambda Functions
 - 8.4 API Gateway
 - 8.5 EventBridge Scheduler
 - 8.6 SNS Notifications
 - 8.7 SES Configuration
- 9. Testing & Verification
- 10. Advantages

1. Project Overview

This project is a modern, fully responsive personal portfolio website built to showcase professional skills, projects, experience, achievements, and certifications. It leverages AWS serverless services to implement:

- Dynamic contact form using AWS Lambda + SES with server-side validation, CORS support, honeypot spam protection, and configurable allowed origins
- Event-driven website lifecycle automation using AWS Lambda + EventBridge Scheduler, demonstrating on-demand infrastructure activation with a 30-minute session window
- SNS email notifications to alert the portfolio owner whenever the website is accessed (best-effort, non-fatal)
- Idempotent upsert scheduling that extends the session window when multiple visitors arrive, preventing orphaned EventBridge schedules

The frontend is a single HTML file (portfolio.html) with inline CSS and vanilla JavaScript — no frameworks, no build tools, no dependencies beyond Google Fonts and Font Awesome.

2. Objectives

- Showcase a professional portfolio with a modern, interactive UI (dark/light mode, parallax scrolling, 4 distinct on-scroll animations, typewriter effects)
- Implement serverless backend: contact form (Lambda + SES) and event-driven website lifecycle (Lambda + EventBridge)
- Demonstrate event-driven lifecycle automation: on-demand infrastructure activation and automated teardown using serverless orchestration
- Notify the portfolio owner via SNS email each time the website is accessed
- Maintain security using IAM least-privilege roles, SES verified identities, server-side input validation, and honeypot-based spam protection
- Use idempotent upsert scheduling to prevent orphaned EventBridge schedules

3. Project Structure

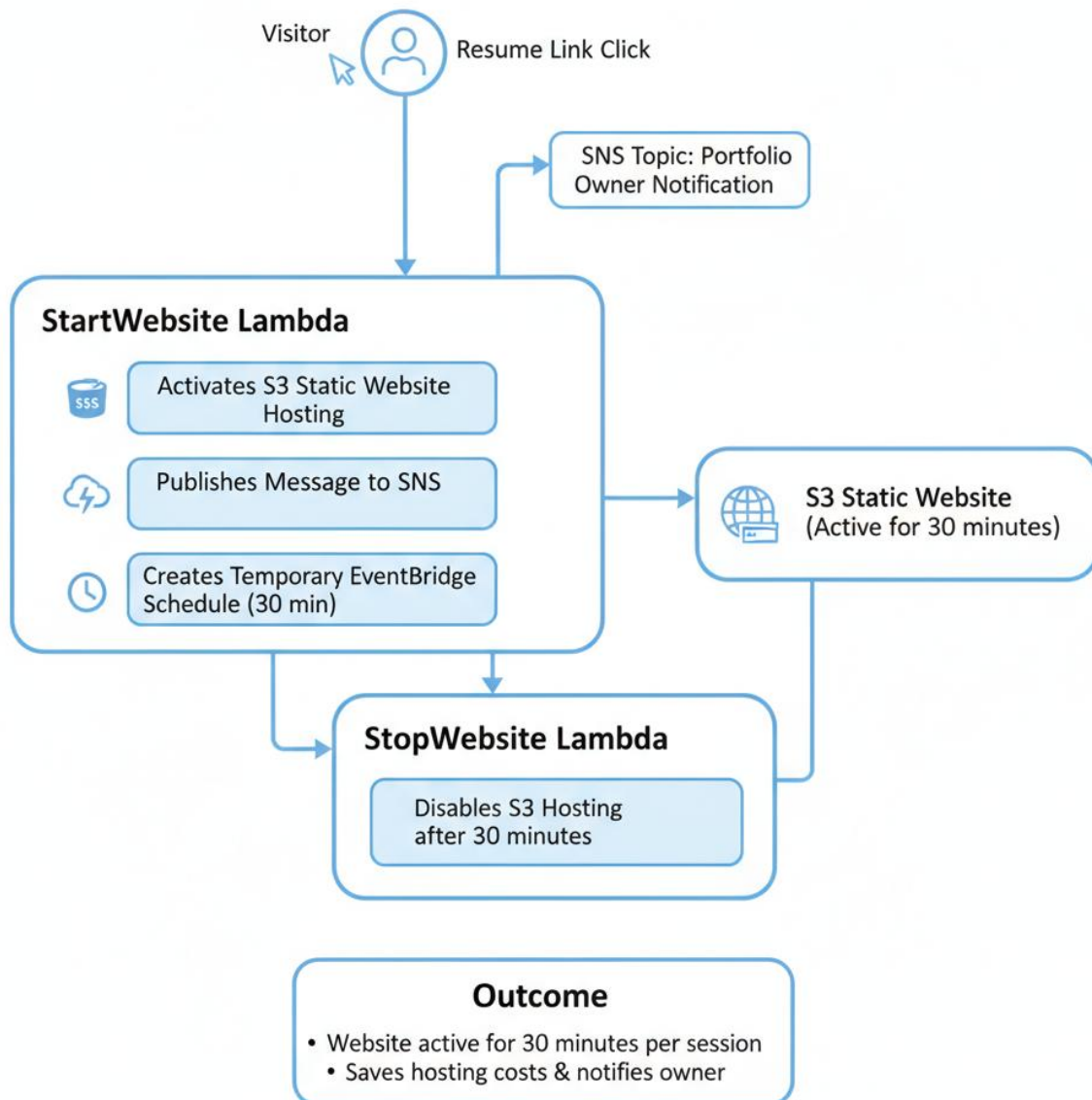
```
On-Demand-Portfolio/  
├─ portfolio.html      # Single-file frontend (HTML + CSS + JS)  
├─ sameer.webp         # Profile photo (WebP optimized)  
├─ sameer_resume.pdf   # Resume PDF (embedded in portfolio)  
├─ *.webp              # Project screenshots (WebP)  
├─ award.webp, Thub.webp, ... # Achievement photos (WebP)  
└─ vercel.json         # Vercel rewrite: / -> /portfolio.html
```

```
|— robots.txt                # Search engine crawling rules
|— README.md                 # Project documentation
|— .gitignore                # Git exclusions
|— OnDemandPortfolio_Documentation.docx # This document
|— OnDemandPortfolio_Documentation.pdf  # PDF export of this document
|— backend/                  # AWS Lambda functions & architecture
|   |— Start_website_lambda.py # Activates S3, schedules stop, sends SNS
|   |— Stop_website_lambda.py  # Removes S3 public policy
|   |— send_messages_SES_lambda.py # Contact form handler (SES + validation)
|   |— architecture_diagram.png # Overall architecture diagram
|   |— website_workflow.png    # Website activation flow diagram
|   |— contact_form_workflow.png # Contact form flow diagram
|   |— README.md               # Backend-specific documentation
|   └─ OnDemandPortfolio_Documentation.* # Documentation copies
└─ architecture/              # Architecture diagrams (duplicate set)
    |— architecture_diagram.png
    |— website_workflow.png
    └─ contact_form_workflow.png
```

All Lambda source code lives in the backend/ folder. The architecture/ folder contains copies of the workflow diagrams for convenience.

4. Architecture & Workflow

4.1 Website Activation Flow



Step 1: Visitor clicks the Start API link (API Gateway GET /start)

Step 2: StartWebsite Lambda executes:

- Applies a public-read S3 bucket policy, making the static site accessible
- Creates or updates a fixed-name EventBridge schedule (upsert pattern) to trigger StopWebsite in 30 minutes

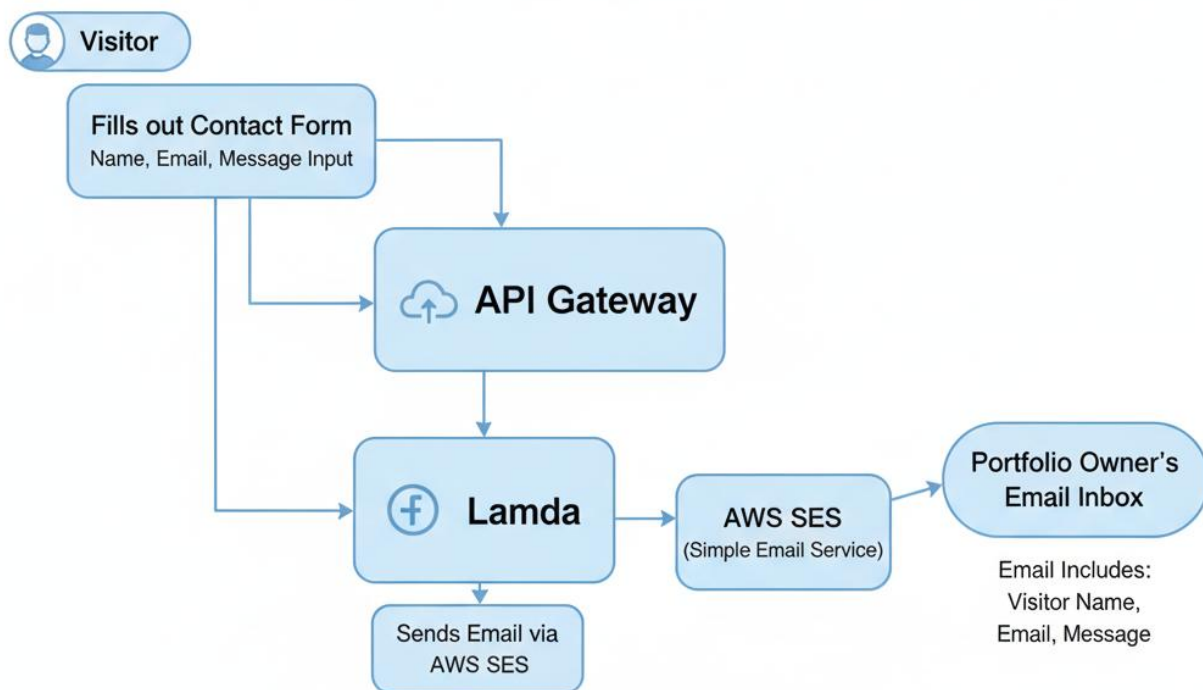
- Publishes an SNS notification to the portfolio owner (best-effort, non-fatal — wrapped in try/except)
- Returns a 302 redirect to the S3 website URL

Step 3: After 30 minutes, EventBridge triggers StopWebsite Lambda, which removes the S3 bucket policy — making the site private again (403 Forbidden).

Upsert pattern: If the schedule already exists (ConflictException), it is updated rather than duplicated. Each new visitor extends the window by 30 more minutes from now, instead of creating orphaned one-off schedules with random names.

4.2 Contact Form Flow

Contact Form Flow



Step 1: Visitor fills out the contact form (name, email, message). A hidden honeypot field ('company') is present but invisible to humans.

Step 2: Client-side JavaScript validates required fields, then sends a POST request to API Gateway (/contact).

Step 3: API Gateway triggers the SendMessageSES Lambda function.

Step 4: Lambda performs server-side validation:

- Checks honeypot: if the hidden 'company' or 'website' field is filled, returns fake success (bot blocked)
- Validates required fields (name, email, message) and email format
- Enforces field length limits (name: 100, email: 254, message: 5000 chars)

Step 5: Lambda sends a formatted email via AWS SES to the portfolio owner, with the visitor's email set as the Reply-To address.

Step 6: Lambda returns a JSON response with CORS headers (Access-Control-Allow-Origin from ALLOWED_ORIGIN env var).

5. AWS Services Used

Service	Purpose	Key Configurations
Amazon S3	Static hosting for portfolio files	Bucket policy toggled dynamically by Lambda; not manually set
AWS Lambda	3 functions: StartWebsite, StopWebsite, SendMessageSES	Python 3.10 runtime; all config via environment variables
EventBridge Scheduler	Automated stop trigger after 30 minutes	Fixed schedule name 'stop-portfolio-website'; upsert pattern
API Gateway (HTTP API)	HTTP endpoints: GET /start, POST /contact	CORS enabled; URL: https://2dubub6pbj.execute-api.us-east-1.amazonaws.com
Amazon SES	Sends contact form emails to portfolio owner	Verified identity required; sandbox vs production mode
Amazon SNS	Visit notification to portfolio owner	Topic: PortfolioSiteStarted; email subscription
AWS IAM	Access control for all services	Separate least-privilege role per Lambda function
CloudWatch	Logging and monitoring	Automatic Lambda execution logs

6. Security & Architectural Rationale

Security

- IAM: Separate roles per Lambda (SiteControlLambdaRole, EventSchedulerInvokeRole, MessageLambdaRole) with least-privilege inline policies scoped to specific resources

- No hardcoded credentials: All ARNs, bucket names, and topic ARNs stored in Lambda environment variables
- SES verified identity: Only authorized senders can trigger emails
- Honeypot spam protection: Hidden form fields catch bots; Lambda also validates server-side
- Input validation: Server-side checks for required fields, email format, and max field lengths
- CORS: SendMessageSES Lambda returns configurable Access-Control-Allow-Origin header (ALLOWED_ORIGIN env var)
- SNS best-effort: Notification is wrapped in try/except — never blocks the visitor redirect
- Idempotent StopWebsite: Handles NoSuchBucketPolicy gracefully (already private = no error)

Why the On-Demand Pattern?

Honestly, S3 static hosting is already dirt cheap — so the on-demand pattern isn't really about saving money here. The point was to demonstrate event-driven lifecycle automation using real AWS services.

In production scenarios with EC2 instances, ECS tasks, or other compute-heavy workloads, this same start/stop pattern can save serious money by eliminating idle resource charges. This project applies that architectural thinking in a simplified context.

It's a portfolio, but it's also a working demo of:

- Serverless orchestration (Lambda → EventBridge → Lambda)
- Automated scheduling with idempotent upsert (create-or-update the same schedule)
- Infrastructure state management (toggling S3 bucket policies programmatically)
- Event-driven notification pipelines (SNS on lifecycle events)
- Graceful degradation (non-fatal SNS, idempotent stop)

7. Features

Backend

- Event-driven lifecycle: on-demand S3 activation with automatic 30-minute teardown
- Idempotent upsert scheduling (ConflictException → update_schedule)
- Contact form: server-side validation, honeypot, CORS headers, ReplyTo support, field length limits
- SNS email notifications (best-effort, non-fatal)
- Configurable CORS origin via ALLOWED_ORIGIN environment variable
- CORS preflight (OPTIONS) handling in Lambda

Frontend

- Single-file HTML with inline CSS and vanilla JavaScript
- Responsive design: breakpoints at 480px, 768px, 992px

- Dark/Light theme toggle with localStorage persistence
- Typewriter animation for professional roles
- Parallax scrolling with gradient orbs (3 layers, different scroll speeds)
- 4 distinct on-scroll animation types (fade-scale, slide-left, slide-right, scale-up)
- Scroll progress indicator bar
- Cover-flow horizontal project showcase with snap scrolling
- Interactive card hover effects with mouse-follow glow
- WebP image optimization (70% total payload reduction vs PNG)
- Accessibility: prefers-reduced-motion, skip-to-content, semantic HTML, ARIA labels
- SEO: Open Graph tags, Twitter Cards, meta descriptions, heading hierarchy
- Sections: Hero, About, Skills, Experience, Achievements, Education, Projects, Coding Portfolio, Certifications, Resume, Contact

8. Implementation Steps

8.1 S3 Setup

1. AWS Console → S3 → Create Bucket (e.g., sameer-portfolio-site)
2. Properties → Static Website Hosting → Enable → Index document: portfolio.html
3. Upload portfolio files: portfolio.html, *.webp images, sameer_resume.pdf
4. Do NOT manually set a bucket policy — StartWebsite Lambda manages this dynamically

When the site is stopped, no public policy exists. Direct S3 access returns 403 Forbidden.

8.2 IAM Roles & Policies

Role 1: SiteControlLambdaRole

Used by: StartWebsite Lambda and StopWebsite Lambda

- Trusted entity: lambda.amazonaws.com
- Managed policy: AWSLambdaBasicExecutionRole

Inline Policy — SiteControlPermissions:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "S3BucketPolicyControl",
      "Effect": "Allow",
      "Action": ["s3:PutBucketPolicy", "s3:DeleteBucketPolicy"],
      "Resource": "arn:aws:s3:::<YOUR_BUCKET_NAME>"
    }
  ]
}
```

```

    },
    {
      "Sid": "EventBridgeSchedulerControl",
      "Effect": "Allow",
      "Action": [
        "scheduler:CreateSchedule",
        "scheduler:UpdateSchedule",
        "scheduler>DeleteSchedule"
      ],
      "Resource": "arn:aws:scheduler*:<YOUR_ACCOUNT_ID>:schedule/default/stop-portfolio-website"
    },
    {
      "Sid": "PassSchedulerRole",
      "Effect": "Allow",
      "Action": "iam:PassRole",
      "Resource": "arn:aws:iam::<YOUR_ACCOUNT_ID>:role/EventSchedulerInvokeRole"
    },
    {
      "Sid": "SNSPublish",
      "Effect": "Allow",
      "Action": "sns:Publish",
      "Resource": "arn:aws:sns*:<YOUR_ACCOUNT_ID>:PortfolioSiteStarted"
    }
  ]
}

```

Role 2: EventSchedulerInvokeRole

Used by: EventBridge Scheduler to invoke StopWebsite Lambda

- Trusted entity: scheduler.amazonaws.com

Trust Policy:

```

{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Principal": { "Service": "scheduler.amazonaws.com" },
    "Action": "sts:AssumeRole"
  }]
}

```

Inline Policy — InvokeStopLambda:

```

{
  "Version": "2012-10-17",
  "Statement": [{

```

```

        "Effect": "Allow",
        "Action": "lambda:InvokeFunction",
        "Resource": "arn:aws:lambda:*:<YOUR_ACCOUNT_ID>:function:StopWebsite"
    }]
}

```

Role 3: MessageLambdaRole

Used by: SendMessageSES Lambda function

- Trusted entity: lambda.amazonaws.com
- Managed policy: AWSLambdaBasicExecutionRole

Inline Policy — SESAllowSend:

```

{
    "Version": "2012-10-17",
    "Statement": [{
        "Effect": "Allow",
        "Action": "ses:SendEmail",
        "Resource": "*"
    }]
}

```

*SES SendEmail requires Resource: * because the caller identity is verified at the SES level, not the IAM resource level.*

8.3 Lambda Functions

All Lambda source files are in the backend/ folder.

Function 1: StartWebsite (backend/Start_website_lambda.py)

Runtime: Python 3.10 | Role: SiteControlLambdaRole | Timeout: 15s

```

import boto3, json, os
from datetime import datetime, timedelta, timezone

sns = boto3.client('sns')
s3 = boto3.client('s3')
scheduler = boto3.client('scheduler')

BUCKET = os.environ['BUCKET']
SITE_URL = os.environ['SITE_URL']
STOP_LAMBDA_ARN = os.environ['STOP_LAMBDA_ARN']
SCHEDULER_ROLE_ARN = os.environ['SCHEDULER_ROLE_ARN']
SNS_TOPIC_ARN = os.environ.get('SNS_TOPIC_ARN')

# Fixed schedule name: upsert pattern prevents orphaned schedules.

```

```

SCHEDULE_NAME = "stop-portfolio-website"

def lambda_handler(event, context):
    # Make site public
    policy = {
        "Version": "2012-10-17",
        "Statement": [{
            "Sid": "PublicReadGetObject",
            "Effect": "Allow",
            "Principal": "*",
            "Action": ["s3:GetObject"],
            "Resource": [f"arn:aws:s3::{BUCKET}/*"]
        }]
    }
    s3.put_bucket_policy(Bucket=BUCKET, Policy=json.dumps(policy))

    # (Re)schedule stop for 30 min from now.
    # Create if new, update if exists (ConflictException = upsert).
    run_time = datetime.now(timezone.utc) + timedelta(minutes=30)
    params = dict(
        Name=SCHEDULE_NAME,
        ScheduleExpression=f"at({run_time.strftime('%Y-%m-%dT%H:%M:%S')})",
        FlexibleTimeWindow={'Mode': 'OFF'},
        Target={'Arn': STOP_LAMBDA_ARN, 'RoleArn': SCHEDULER_ROLE_ARN},
    )
    try:
        scheduler.create_schedule(**params)
    except scheduler.exceptions.ConflictException:
        scheduler.update_schedule(**params)

    # SNS is best-effort: never let it break the redirect.
    if SNS_TOPIC_ARN:
        try:
            sns.publish(
                TopicArn=SNS_TOPIC_ARN,
                Message='Someone just started your portfolio website!',
                Subject='Portfolio Notification'
            )
        except Exception as e:
            print(f"SNS publish failed (non-fatal): {e}")

    return {
        "statusCode": 302,
        "headers": {"Location": SITE_URL},
        "body": json.dumps({"message": "Redirecting"})
    }

```

```
}
```

Environment Variables:

Variable	Description
BUCKET	S3 bucket name (e.g., sameer-portfolio-site)
SITE_URL	S3 website endpoint URL to redirect visitors to
STOP_LAMBDA_ARN	ARN of the StopWebsite Lambda function
SCHEDULER_ROLE_ARN	ARN of EventSchedulerInvokeRole
SNS_TOPIC_ARN	ARN of SNS topic (optional — if unset, notifications are skipped)

Function 2: StopWebsite (backend/Stop_website_lambda.py)

Runtime: Python 3.10 | Role: SiteControlLambdaRole | Timeout: 10s

```
import boto3, os, json
from botocore.exceptions import ClientError

s3 = boto3.client('s3')
BUCKET = os.environ.get('BUCKET')

def lambda_handler(event, context):
    try:
        s3.delete_bucket_policy(Bucket=BUCKET)
        print(f"Public read policy removed from {BUCKET}; website is now private.")
    except ClientError as e:
        code = e.response['Error']['Code']
        # Already private / no policy = idempotent success
        if code in ('NoSuchBucketPolicy', 'NoSuchBucket'):
            print(f"No policy to remove ({code}); bucket already private.")
        else:
            print(f"Failed to remove bucket policy: {code}")
            raise
    return {"statusCode": 200, "body": json.dumps({"message": "Website stopped"})}
```

Environment Variables:

Variable	Description
BUCKET	S3 bucket name (same as StartWebsite)

Function 3: SendMessageSES (backend/send_messages_SES_lambda.py)

Runtime: Python 3.10 | Role: MessageLambdaRole | Timeout: 10s

```

import boto3
import json
import os
from botocore.exceptions import ClientError

ses = boto3.client('ses', region_name='us-east-1')

FROM = os.environ['FROM'] # Must be verified in SES
TO = os.environ['TO']      # Recipient email address
ALLOWED_ORIGIN = os.environ.get('ALLOWED_ORIGIN', '*')

# Abuse limits
MAX_NAME = 100
MAX_EMAIL = 254
MAX_MESSAGE = 5000

def _resp(status, payload):
    return {
        'statusCode': status,
        'headers': {
            'Content-Type': 'application/json',
            'Access-Control-Allow-Origin': ALLOWED_ORIGIN,
            'Access-Control-Allow-Headers': 'Content-Type',
            'Access-Control-Allow-Methods': 'POST, OPTIONS',
        },
        'body': json.dumps(payload),
    }

def lambda_handler(event, context):
    # Handle CORS preflight
    if (event.get('httpMethod') or '').upper() == 'OPTIONS':
        return _resp(200, {'message': 'ok'})

    try:
        body = json.loads(event.get('body') or '{}')

        # Honeypot: bots fill hidden fields. Pretend success, send nothing.
        if body.get('company') or body.get('website'):
            return _resp(200, {'message': 'Message sent successfully!'})

        name = (body.get('name') or '').strip()
        email = (body.get('email') or '').strip()
        message = (body.get('message') or '').strip()

```

```

# Server-side validation
if not name or not email or not message:
    return _resp(400, {'error': 'Name, email and message are required.'})
if '@' not in email or '.' not in email.split('@')[-1]:
    return _resp(400, {'error': 'Please provide a valid email address.'})
if len(name) > MAX_NAME or len(email) > MAX_EMAIL or len(message) > MAX_MESSAGE:
    return _resp(400, {'error': 'One or more fields exceed the allowed length.'})

subject = f"Portfolio Message from {name}"
email_body = f"Name: {name}\nEmail: {email}\n\nMessage:\n{message}"

ses.send_email(
    Source=FROM,
    Destination={'ToAddresses': [TO]},
    ReplyToAddresses=[email] if '@' in email else [],
    Message={
        'Subject': {'Data': subject},
        'Body': {'Text': {'Data': email_body}},
    },
)

return _resp(200, {'message': 'Message sent successfully!'})

except ClientError as e:
    print(f"SES Error: {e.response['Error']['Message']}")
    return _resp(500, {'error': 'Failed to send message via SES'})

except Exception as e:
    print(f"Error: {str(e)}")
    return _resp(500, {'error': 'Internal server error'})

```

Environment Variables:

Variable	Description
FROM	Verified sender email address in SES
TO	Recipient email address (portfolio owner)
ALLOWED_ORIGIN	CORS origin (default: * ; set to your domain in production)

If SES is in sandbox mode, both FROM and TO must be verified. Request production access in the SES console to send to unverified recipients.

8.4 API Gateway

1. AWS Console → API Gateway → Create HTTP API

2. Add integrations:

Route	Method	Integration
/start	GET	StartWebsite Lambda
/contact	POST	SendMessageSES Lambda

3. CORS Configuration:

- Allowed Origins: * (or your specific domain)
- Allowed Methods: GET, POST, OPTIONS
- Allowed Headers: Content-Type

4. Deploy to \$default stage

5. Invoke URL: <https://2dubub6pbj.execute-api.us-east-1.amazonaws.com>

The /contact route also handles OPTIONS preflight. The Lambda returns CORS headers in every response.

8.5 EventBridge Scheduler

EventBridge Scheduler is NOT configured manually. StartWebsite Lambda creates and manages the schedule programmatically.

Design decisions:

- Fixed name: stop-portfolio-website (not random UUIDs — prevents orphan pile-up)
- Upsert: create_schedule on first visit, update_schedule on subsequent (ConflictException catch)
- Expression: at(YYYY-MM-DDTHH:MM:SS) — one-shot, fires exactly once
- FlexibleTimeWindow: OFF — fires at exact scheduled time
- Target: StopWebsite Lambda via EventSchedulerInvokeRole

8.6 SNS Notifications

1. AWS Console → SNS → Create Topic → Name: PortfolioSiteStarted
2. Create Email Subscription → enter your email → confirm in inbox
3. Copy Topic ARN → set as SNS_TOPIC_ARN env var in StartWebsite Lambda

SNS is entirely optional. If SNS_TOPIC_ARN is not set, notifications are skipped. If publishing fails, the error is logged but the redirect is never blocked.

8.7 SES Configuration

1. AWS Console → SES → Verified Identities → Create Identity
2. Verify your email address (check inbox for verification link)
3. Sandbox mode: also verify the recipient (TO) email address

4. Production: request production access (Account Dashboard → Request Production Access)

In sandbox mode, the contact form only works when both FROM and TO are verified. Production mode allows sending to any email address.

9. Testing & Verification

Test	Steps	Expected Result
Website Activation	Click Start API URL in browser	302 redirect to S3 site; portfolio loads
SNS Notification	Check email after clicking Start API	Email: 'Someone just started your portfolio website!'
30-Minute Auto-Stop	Wait 30 min, then access S3 endpoint directly	403 Forbidden (policy removed)
Session Extension	Click Start API while site is already up	Schedule updated (not duplicated); timer resets
EventBridge Schedule	Check EventBridge → Schedules in console	Single schedule named 'stop-portfolio-website'
Contact Form	Submit the contact form on the portfolio	Email via SES with name, email, message
Honeypot	Submit with hidden 'company' field filled	Fake success; no email sent
Input Validation	Submit with empty fields or bad email	400 error with descriptive message
CORS Headers	Check /contact POST response headers	Access-Control-Allow-Origin present
CloudWatch	Check Log Groups for all 3 Lambdas	Execution logs with timestamps

10. Advantages

- **Event-Driven Architecture:** Demonstrates real serverless orchestration — Lambda triggers EventBridge, EventBridge triggers Lambda. The same start/stop pattern applies to EC2, ECS, and other compute-heavy workloads where idle costs are significant.
- **Serverless & Scalable:** Lambda, EventBridge, and SES handle all backend logic without provisioned servers.
- **Recruiter-Friendly:** Anyone can access and interact without an AWS account.
- **Secure:** IAM least-privilege, SES verified identities, honeypot + server-side validation, no hardcoded secrets.
- **Idempotent:** Upsert scheduling prevents duplicate schedules; StopWebsite handles already-private buckets gracefully.
- **Observable:** CloudWatch logs for every Lambda invocation; EventBridge schedule visible in console.
- **Maintainable:** Modular serverless architecture. Single HTML file for the frontend. No build tools or frameworks.

